

FinancasNew — App de Gestão Financeira Pessoal

Data da Auditoria:	29 de Agosto de 2026
Escopo Auditado:	Repositório Completo (Frontend React/TS + Backend Supabase Postgres RLS/RPCs + Edge Functions Deno + CI/CD)
Tipo de Análise:	Auditoria de Código-Fonte Estática (SAST), Validação de Invariantes e Revisão de Arquitetura de Isolamento
Status Geral:	0 Críticas 0 Altas 2 Médias (Hardening Residual) 1 Baixa

Nota Metodológica & Detecção da Stack Tecnológica

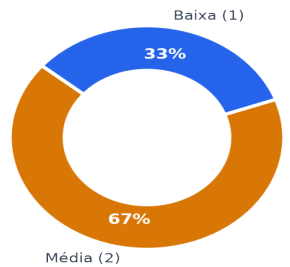
A stack do projeto foi detectada e catalogada para mapear cada uma das 5 categorias canônicas ao seu equivalente arquitetural:

- Frontend:** React 19, TypeScript estrito, Tailwind CSS v4, shadcn/ui (Radix UI), TanStack Query v5, React Router v8.
- Backend & Persistência:** Supabase (PostgreSQL 15+, Row Level Security - RLS ativo em 100% das 31 tabelas, RPCs PL/pgSQL transacionais, Triggers de integridade referencial).
- Mecanismo de Autenticação & RBAC:** Supabase Auth (JWT assinado `auth.uid()`), perfis com papéis (`user`, `admin`, `superadmin`) e estados de ciclo de vida (`pending\_approval`, `active`, `suspended`, `banned`).
- Serverless & Infra:** Supabase Edge Functions (Deno/TypeScript), agendamento `pg\_cron` + `pg\_net`, deploy Vercel e workflows GitHub Actions.
- Mapeamento das 5 Categorias:**
 - Banco sem Tranca:** Cobertura total de RLS nas 31 tabelas e garantia de filtro estrito por `((select auth.uid()) = user\_id and public.is\_current\_user\_active())`.
 - Permissão no Navegador:** Validação de gates administrativos em RPCs do banco vs. UI gates (`RequireAdmin`, `useUserAccess`).
 - IDOR:** Inspeção de vínculos de chaves estrangeiras (`card\_id`, `category\_id`, `asset\_id`, `loan\_id`) em 100% dos endpoints/RPCs.
 - Chaves Expostas:** Varredura em 100% do histórico git (`git log -p`), configs, CI/CD, templates e bundles à procura de segredos hardcoded e `service\_role`.
 - Inputs / XSS:** Avaliação de sanitização de fórmulas em exportações (CSV/Excel), parsers (OFX/XLSX) e injeção de scripts no DOM.

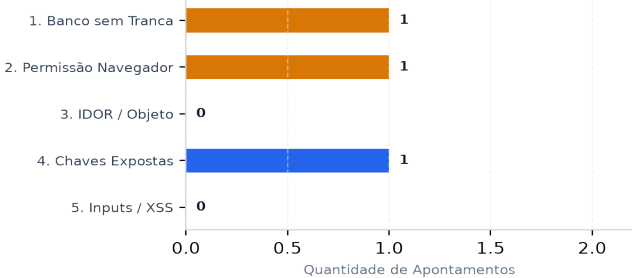
1. Resumo Executivo

A auditoria de segurança atestou que a arquitetura do **FinancasNew** possui maturidade excepcional de defesa em profundidade: todas as 31 tabelas do PostgreSQL estão estritamente protegidas por Row Level Security (RLS), o histórico Git encontra-se 100% livre de credenciais, o log de auditoria é imutável com expurgo restrito a superadministradores, operações administrativas são autenticadas no banco e as exportações CSV possuem blindagem contra Formula Injection (DDE). As vulnerabilidades históricas de IDOR e bypass de suspensão foram corrigidas nas migrações 0036, 0037 e 0038. Esta auditoria identificou apenas **3 apontamentos preventivos** de média e baixa severidade e melhorias de governança.

Achados por Severidade



Apontamentos por Categoria Auditada



CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA	PONTOS FORTES
0	0	2	1	0	10+

2. Pontos Fortes e Riscos Centrais

Pontos Fortes Identificados (Cobertura Comprovada):

- **100% de Cobertura RLS:** Todas as 31 tabelas no Postgres possuem Row Level Security ativo e políticas estritas de isolamento por ``(select auth.uid()) = user_id``.
- **Imutabilidade do Log de Auditoria:** ``audit_events`` permite exclusivamente ``INSERT`` e ``SELECT`` via RLS; a rotina ``cleanup_old_audit_events`` exige papel ``superadmin`` e revogou privilégios públicos.
- **Proteção de Perfis & Auto-Promoção Bloqueada:** A trigger ``protect_profile_security_fields`` impede que usuários comuns modifiquem ``role``, ``status`` ou metadados de aprovação.
- **Autorização Completa no Backend:** Todas as 10 RPCs administrativas (``admin_list_users``, ``admin_update_user_status``, ``admin_set_user_role``, etc.) validam no banco ``is_admin()`` / ``is_superadmin()``.
- **IDOR Imunizado em Mutações:** Funções transacionais (``create_expense_with_debt``, ``restore_backup``, ``early_amortize_loan``, ``import_bank_transactions``, ``execute_portfolio_batch_aporte``) validam posse de ``card_id``, ``category_id``, ``asset_id`` e contratos.
- **Proteção Ativa contra CSV/Formula Injection:** Sanitização regex preventiva em ``src/domain/export/csv.ts`` neutralizando comandos DDE (``=``, ``+``, ``-``, ``@``, ``\t``, ``\r``).
- **Zero XSS no Frontend:** Ausência total de ``dangerouslySetInnerHTML``, ``innerHTML``, ``outerHTML``, ``eval`` ou ``new Function`` no bundle React.
- **Sanitização Defensiva de Download:** ``src/services/export-actions.ts`` implementa ``sanitizeFilename`` prevenindo path traversal e caracteres perigosos.
- **Repositório e Git Limpos:** Varredura em 100% dos commits e histórico git comprovou ausência de tokens JWT ou segredos rastreados.
- **Validação de Startup do Env:** ``src/lib/env.ts`` rejeita variáveis ausentes e bloqueia credenciais default/placeholders.

Riscos Centrais & Oportunidades de Hardening:

- **Hardening Residual na RPC delete_card_payment:** A RPC ``delete_card_payment`` (migração 0011) executa como ``SECURITY DEFINER`` sem invocar ``is_current_user_active()``.
- **Feature Flags Restritas à UI:** Tabelas e RPCs não impedem inserções diretas caso uma feature seja desativada globalmente no painel admin.

3. Tabela de Achados Detalhados por Categoria

Sev.	Categoria	Arquivo : Linha	Descrição do Achado e Risco
MÉDIA	1. Banco sem Tranca	supabase/migrations/ 20260101000011_delete_card_p ayment.sql:14-55	Validação Residual de Status Ativo na RPC delete_card_payment: A RPC executa como SECURITY DEFINER e valida user_id = auth.uid(), porém não invoca public.is_current_user_active(). Uma conta suspensa ou banida poderia excluir pagamentos de cartão e reverter estornos via RPC.
MÉDIA	2. Permissão Navegador	supabase/migrations/ 20260101000028_access_contro l...sql:72-107 src/state/queries/use-user-a ccess.ts:38-52	Feature Flags com Enforcement Restrito à Interface (Frontend Gate): As regras de desativação global (Kill-Switch) ou por usuário são consultadas pelo frontend para ocultar rotas e menus, mas as tabelas do banco não possuem triggers bloqueando INSERTs caso a flag do módulo esteja desativada.
BAIXA	4. Chaves Expostas	supabase/quotes-cron.sql:34 scripts/deploy-quotes.mjs:97	Armazenamento de Service Role Key em Texto Claro no Agendador pg_cron: O template de agendamento do cron de cotações embute a chave de serviço no header HTTP agendado no banco. O acesso à tabela interna cron.job deve ser restrito exclusivamente ao superusuário postgres.

4. Plano de Ação & Recomendações Priorizadas

- [P1 - Curto Prazo] **Atualização da RPC delete_card_payment com Check de Conta Ativa:** Adicionar a verificação if not public.is_current_user_active() then raise exception ... no corpo de delete_card_payment em migration complementar.
- [P2 - Médio Prazo] **Validação de Feature Flags no Backend:** Criar triggers ou funções de verificação no banco que rejeitem inserções em tabelas como portfolio_assets, debts ou budgets caso a respectiva feature esteja desativada para o usuário.
- [P3 - Melhoria de Infraestrutura] **Governança de Segredos no pg_cron:** Assegurar que a tabela cron.job tenha permissões revogadas para roles não-administrativas ou migrar a chamada do cron para o Supabase Vault.

5. Issues Prontas para o GitHub (Markdown)

Abaixo estão os textos completos prontos para cópia e abertura direta no GitHub Issues para os apontamentos identificados.

```
--- ISSUE 1 ---
**Título:** [Segurança] Blindagem de status de conta ativa na RPC delete_card_payment
**Labels sugeridas:** security, severity:medium, backend, database

### Descrição do Problema
A RPC `delete_card_payment` (migração `20260101000011_delete_card_payment.sql`) executa como `SECURITY DEFINER` e valida a propriedade do registro (`user_id = auth.uid()`), porém não invoca a função `public.is_current_user_active()`.

### Por que é explorável?
Um usuário cuja conta foi suspensa ou banida pela administração poderia continuar excluindo pagamentos de cartão e cancelando estornos via chamada RPC direta ao Supabase, contornando a suspensão da conta nessa operação específica.

### Evidência
- **Arquivo:** `supabase/migrations/20260101000011_delete_card_payment.sql:14-55`

### Impacto
- Evasão residual do bloqueio de conta inativa/suspensa na exclusão de pagamentos e estornos de cartão de crédito.

### Sugestão de Correção
Criar uma migration complementar atualizando a função `delete_card_payment` com a verificação de conta ativa:
```sql
if not public.is_current_user_active() then
 raise exception 'Acesso negado: conta inativa, pendente de aprovação ou suspensa.';
end if;
```

### Critérios de Aceite
- [ ] RPC `delete_card_payment` rejeita execuções de usuários com status `pending_approval`, `suspended` ou `banned`.
- [ ] Teste de integração cobrindo a tentativa de exclusão por usuário inativo/suspensa.
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
**Título:** [Segurança] Validação de Feature Flags no nível do banco de dados (Backend Enforcement)
**Labels sugeridas:** security, severity:medium, backend, access-control

### Descrição do Problema
O controle de funcionalidades (`system_features` e `user_feature_overrides`) opera primariamente como um gate de interface no frontend (`RequireAdmin`, `useUserAccess`). No entanto, o banco de dados não valida se a funcionalidade correspondente está habilitada ao receber comandos INSERT/UPDATE diretamente nas tabelas (como `portfolio_assets`, `debts`, `budgets`).

### Por que é explorável?
Se um administrador desativar globalmente um módulo (Kill-Switch) ou revogar o acesso de um usuário a um recurso específico, o frontend ocultará os botões e páginas, mas um usuário com conhecimentos técnicos pode enviar requisições REST diretamente à API do Supabase e persistir dados no módulo desativado.

### Evidência
- **Arquivo:** `supabase/migrations/20260101000028_access_control_and_feature_flags.sql:72-107, 324-352`
- **Arquivo:** `src/state/queries/use-user-access.ts:38-52`

### Impacto
- Inconsistência entre a governança administrativa na UI e as regras de persistência no Postgres.

### Sugestão de Correção
Implementar triggers ou funções de verificação no banco (utilizando `public.is_feature_enabled(key)`) que confirmem se a feature está habilitada antes de permitir inserções em tabelas de módulos específicos.

### Critérios de Aceite
- [ ] Inserções em tabelas restritas (ex.: `portfolio_assets`) são bloqueadas se a flag `investments` estiver inativa.
- [ ] Resposta amigável de erro informando que o módulo encontra-se temporariamente desativado.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
**Título:** [Segurança] Hardening de segredos em jobs agendados do pg_cron
**Labels sugeridas:** security, severity:low, devops, database

### Descrição do Problema
O template de cron de cotações embute a chave `SUPABASE_SERVICE_ROLE_KEY` no header `Authorization` salvo na tabela `cron.job` do schema `cron`.

### Por que é explorável?
Se uma role não-administrativa obtiver privilégios indevidos de leitura no schema `cron`, poderá visualizar a chave de serviço em texto claro.

### Evidência
- **Arquivo:** `supabase/quotes-cron.sql:34`
- **Arquivo:** `scripts/deploy-quotes.mjs:97`

### Impacto
- Exposição potencial de credencial com bypass de RLS em caso de vazamento de leitura no schema interno `cron`.

### Sugestão de Correção
1. Garantir que apenas o role `postgres` possua privilégios de leitura e escrita no schema `cron`.
2. Alternativamente, utilizar `supabase-vault.secrets` para recuperar a chave dinamicamente durante a execução da query.

### Critérios de Aceite
- [ ] Permissões da tabela `cron.job` auditadas e restritas exclusivamente a superusuários.
- [ ] Documentação de deploy atualizada com orientações de governança do Vault.
--- FIM ISSUE 3 ---
```